

FIAP x GoodWe Brasil — EV Challenge

Relatório de Evolução do Projeto — Sprint 03

1. Resumo da evolução

Nas Sprints 1 e 2, o chatbot EV Challenge — GoodWe foi implementado como um pipeline RAG manual: extração e chunking do manual em PDF, indexação em ChromaDB e geração de respostas via HuggingFace Inference Client, com o contexto recuperado injetado diretamente no system prompt a cada pergunta. O parâmetro de histórico existia na função de conversa, mas não era efetivamente utilizado na montagem das mensagens enviadas ao modelo — ou seja, não havia memória de conversa real entre turnos.

Na Sprint 03, o núcleo conversacional foi reconstruído sobre o framework LangChain. O retrieval do ChromaDB passou a ser orquestrado por um vectorstore do LangChain, a montagem do prompt passou a usar templates estruturados (ChatPromptTemplate) e a memória de conversa passou a ser gerenciada nativamente pelo framework, por sessão, através de RunnableWithMessageHistory. Além disso, formalizamos um conjunto de casos de teste de segurança e passamos a comparar sistematicamente dois modelos diferentes sob os mesmos parâmetros.

2. Refatoração — decisões técnicas e trade-offs

A escolha do LangChain se justifica pela integração nativa com os componentes que o grupo já usava (ChromaDB e HuggingFace), minimizando a reescrita necessária e permitindo focar o esforço na peça que realmente faltava: a memória de sessão gerenciada pelo framework, em vez de controlada manualmente.

Principais decisões

- Uso de RunnableWithMessageHistory com um dicionário em memória (InMemoryChatMessageHistory) por session_id, em vez de reimplementar o controle de histórico manualmente.
- Introdução de um modelo de embeddings explícito (sentence-transformers/all-MiniLM-L6-v2) para o vectorstore do LangChain, em vez do embedding default do ChromaDB usado na Sprint 02 — trade-off: uma dependência extra, em troca de um pipeline mais transparente e portátil para outros vectorstores.
- Manutenção do mesmo system prompt/persona da Sprint 01, com adição explícita de regras de guardrail (recusa de conselho jurídico/financeiro/elétrico e resistência a tentativas de troca de persona).
- Pipeline principal (memória + testes de segurança) em modelo local e gratuito (Qwen/Qwen2.5-1.5B-Instruct, com fallback para TinyLlama), para não depender de cota de Inference Providers. A comparação entre modelos (Bloco B) usa a API HuggingFace quando há token/crédito; caso contrário, reaproveita automaticamente o comparativo_modelos.csv já existente com resultados reais de execução anterior.

3. Comparativo antes/depois

Critério	Sprints 1/2 (versão manual/legado)	Sprint 03 (framework de agentes)
Orquestração	Chamadas diretas ao ChromaDB e ao HuggingFace Inference Client, montadas manualmente	LangChain (langchain-chroma + langchain-huggingface), pipeline via LCEL
Memória de conversa	Parâmetro historico existia na função, mas não era usado na montagem das mensagens — sem memória real	RunnableWithMessageHistory, memória por session_id, validada em 3+ turnos
Qualidade das respostas (nota no eval)	Não medida formalmente	Média 4,4 (Modelo A) e 4,6 (Modelo B) — ver relatorio_modelos.md
Tokens/palavras por turno	Não medido formalmente	Média 143 palavras (A) e 127 palavras (B) por resposta
Latência média	Não medida formalmente	13,38 s (Modelo A, DeepSeek-V4.1-Flash) e 12,40 s (Modelo B, Llama-3.1-8B-Instruct)

Testes de segurança	Não formalizados	6 casos documentados (prompt injection, escopo, conselho jurídico/elétrico) — ver resultados_seguranca.csv
---------------------	------------------	--

Os números completos por pergunta estão em comparativo_modelos.csv, e os 6 casos de teste de segurança com a resposta obtida e a avaliação manual de cada um estão em resultados_seguranca.csv, ambos no repositório junto deste relatório.

4. Problemas encontrados e soluções

Problema 1 — memória de conversa inexistente na Sprint 02

Ao revisar o código da Sprint 02 para a migração, identificamos que a função de conversa recebia o histórico da interface, mas nunca o incluía nas mensagens enviadas ao modelo — cada pergunta era respondida sem nenhum contexto das anteriores. Decisão: usar isso como ponto de partida formal do "antes" no comparativo, e resolver com RunnableWithMessageHistory na Sprint 03, validando explicitamente com um teste de 3+ turnos que depende de contexto anterior.

Problema 2 — esgotamento de créditos gratuitos da HuggingFace (erro 402)

Ao rodar a comparação entre modelos e testes iniciais, a cota mensal de créditos gratuitos de Inference Providers se esgotou mais de uma vez (erro HTTP 402), especialmente com modelos maiores. Decisão: (1) o pipeline principal (memória + segurança) passou a rodar em modelo local gratuito, eliminando dependência de cota para o Bloco A e o Bloco C; (2) a comparação entre modelos (Bloco B) ficou restrita a duas variantes leves e fixas (DeepSeek-V4.1-Flash e Llama-3.1-8B-Instruct); (3) o max_new_tokens foi reduzido de 1000 para 300 para conter o consumo; (4) quando não há token ou a cota está esgotada, o notebook reaproveita automaticamente o comparativo_modelos.csv já existente com resultados reais, em vez de travar; (5) o grupo revezou o token entre contas via Kaggle Secrets, sem nunca commitar a chave no repositório. Trade-off assumido: respostas mais longas saíram truncadas em alguns casos (ver relatorio_modelos.md, seção 3), o que o grupo documentou como limitação conhecida em vez de mascarar.

5. Equipe e divisão de trabalho

Nome	RM	Turma	Tarefa principal
Gustavo Bitencourt (líder)	568885	1CCPZ	Coordenação geral, migração do pipeline conversacional para LangChain, memória de sessão
Leonardo Takachi	569066	1CCPZ	Estruturação do pipeline com memória (RunnableWithMessageHistory) e integração do modelo local
Daniel Vieira	573326	1CCPZ	Revisão de código, identificação de erros durante a execução e testes de segurança
Giovane Salazar	570396	1CCPZ	Setup e ingestão do PDF, embeddings/vectorstore, fornecimento do documento e token de API